

Hình 2-1. Giá nhà ở California

Hãy nhìn vào bức tranh toàn cảnh

Chào mừng đến với Tập đoàn Nhà ở Học máy! Nhiệm vụ đầu tiên của bạn là sử dụng dữ liệu điều tra dân số California để xây dựng mô hình giá nhà ở trong tiểu bang. Dữ liệu này bao gồm các chỉ số như dân số, thu nhập trung bình và giá nhà trung bình cho mỗi nhóm khu phố ở California. Nhóm khu phố là đơn vị địa lý nhỏ nhất mà Cục Điều tra Dân số Hoa Kỳ công bố dữ liệu mẫu (một nhóm khu phố thường có dân số từ 600 đến 3.000 người). Tôi sẽ gọi tắt chúng là "quận".

Mô hình của bạn cần học hỏi từ dữ liệu này và có khả năng dự đoán giá nhà ở trung bình tại bất kỳ quận nào, dựa trên tất cả các chỉ số khác.

MẸO

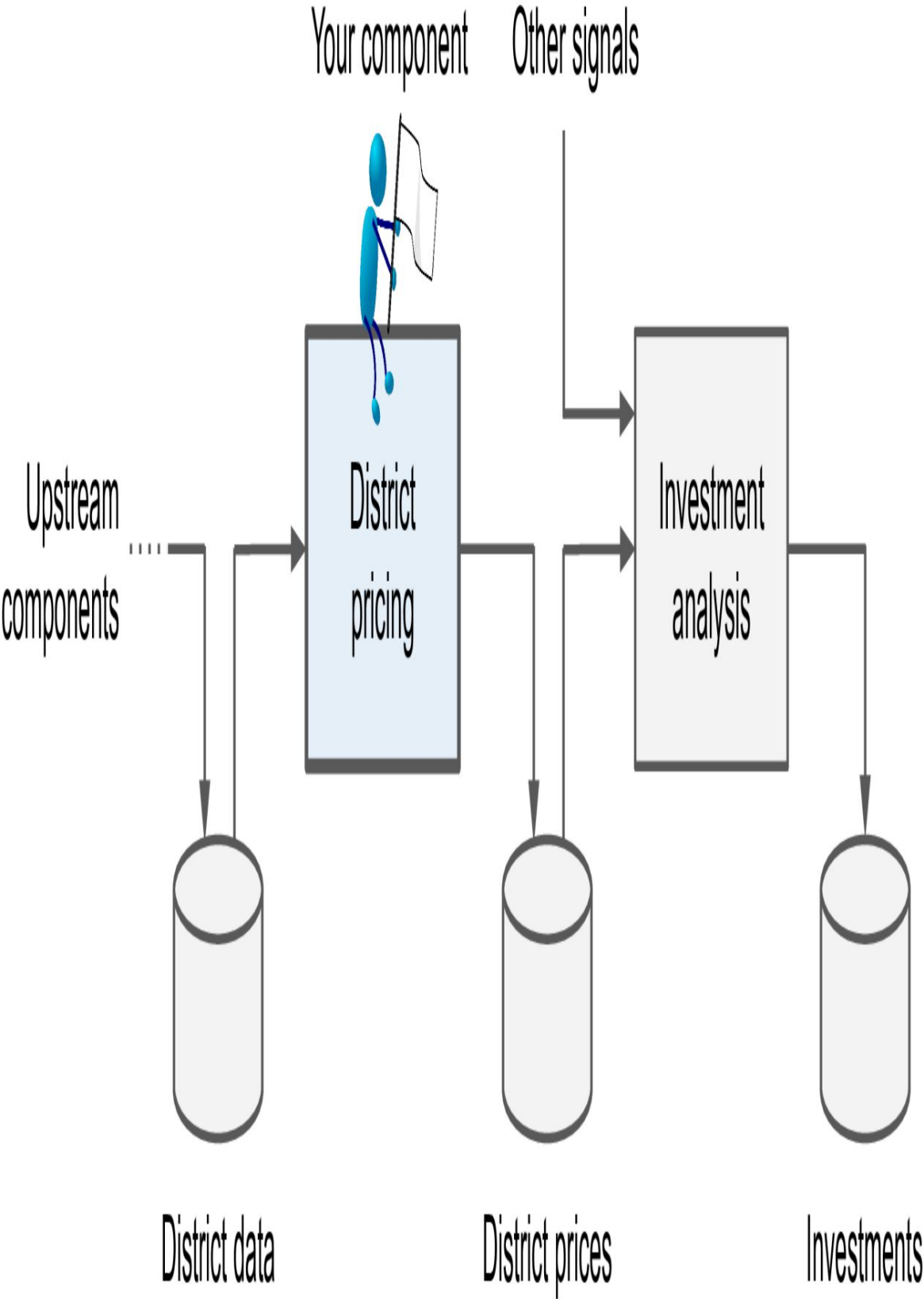
Vì bạn là một nhà khoa học dữ liệu có tổ chức tốt, việc đầu tiên bạn nên làm là lấy ra danh sách kiểm tra dự án Học máy của mình. Bạn có thể bắt đầu với danh sách trong **Phụ lục A**; nó sẽ hoạt động khá tốt cho hầu hết các dự án Học máy, nhưng hãy đảm bảo điều chỉnh nó cho phù hợp với nhu cầu của bạn. Trong chương này, chúng ta sẽ xem xét nhiều mục trong danh sách kiểm tra, nhưng chúng ta cũng sẽ bỏ qua một vài mục, hoặc vì chúng tự giải thích hoặc vì chúng sẽ được thảo luận trong các chương sau.

Xác định vấn đề

Câu hỏi đầu tiên bạn cần hỏi sếp là mục tiêu kinh doanh chính xác là gì. Xây dựng mô hình có lẽ không phải là mục tiêu cuối cùng. Công ty kỳ vọng sẽ sử dụng và thu lợi từ mô hình này như thế nào? Việc xác định mục tiêu rất quan trọng vì nó sẽ quyết định cách bạn định hình vấn đề, thuật toán bạn sẽ chọn, thước đo hiệu suất bạn sẽ sử dụng để đánh giá mô hình và lượng công sức bạn sẽ dành cho việc tinh chỉnh nó.

Sếp của bạn trả lời rằng kết quả đầu ra của mô hình (dự đoán giá nhà trung bình của một quận) sẽ được đưa vào một hệ thống Học máy khác (xem **Hình 2-2**), cùng với nhiều tín hiệu khác. Hệ thống tiếp theo này² sẽ

Xác định xem việc đầu tư vào một lĩnh vực cụ thể có đáng giá hay không. Làm đúng điều này rất quan trọng, vì nó ảnh hưởng trực tiếp đến doanh thu.



Hình 2-2. Quy trình học máy cho đầu tư bất động sản.

ĐƯỜNG ỐNG

Một chuỗi các thành phần xử lý dữ liệu được gọi là đường dẫn dữ liệu (data pipeline). Các pipeline rất phổ biến trong các hệ thống Học máy, vì có rất nhiều dữ liệu cần xử lý và nhiều phép biến đổi dữ liệu cần áp dụng.

Các thành phần thường hoạt động bất đồng bộ. Mỗi thành phần sẽ lấy một lượng lớn dữ liệu, xử lý nó và xuất kết quả vào một kho dữ liệu khác. Sau đó, một thời gian sau, thành phần tiếp theo trong quy trình sẽ lấy dữ liệu này và xuất ra kết quả riêng của nó. Mỗi thành phần khá độc lập: giao diện giữa các thành phần chỉ đơn giản là kho dữ liệu. Điều này giúp hệ thống dễ nắm bắt (với sự trợ giúp của biểu đồ luồng dữ liệu), và các nhóm khác nhau có thể tập trung vào các thành phần khác nhau.

Hơn nữa, nếu một thành phần bị hỏng, các thành phần tiếp theo thường có thể tiếp tục hoạt động bình thường (ít nhất là trong một thời gian) chỉ bằng cách sử dụng đầu ra cuối cùng từ thành phần bị hỏng. Điều này làm cho kiến trúc trở nên khá mạnh mẽ.

Mặt khác, một linh kiện bị hỏng có thể không được phát hiện trong một thời gian nếu không có hệ thống giám sát thích hợp. Dữ liệu trở nên lỗi thời và hiệu suất tổng thể của hệ thống giảm sút.

Câu hỏi tiếp theo bạn cần hỏi sếp là giải pháp hiện tại (nếu có) trông như thế nào. Tình hình hiện tại thường sẽ cung cấp cho bạn một điểm tham chiếu về hiệu suất, cũng như những hiểu biết về cách giải quyết vấn đề. Sếp bạn trả lời rằng giá nhà ở trong khu vực hiện đang được các chuyên gia ước tính thủ công: một nhóm thu thập thông tin cập nhật về khu vực đó, và khi họ không thể có được giá nhà ở trung bình, họ sẽ ước tính bằng cách sử dụng các quy tắc phức tạp.

Việc này tốn kém và mất thời gian, và các ước tính của họ không chính xác; trong trường hợp họ tìm ra được giá nhà trung bình thực tế, họ thường nhận ra rằng ước tính của mình sai lệch hơn 30%. Đó là lý do tại sao công ty cho rằng việc huấn luyện một mô hình để dự đoán giá nhà trung bình của một quận, dựa trên các dữ liệu khác về quận đó, sẽ rất hữu ích. Dữ liệu điều tra dân số có vẻ là một tập dữ liệu tuyệt vời để khai thác cho mục đích này, vì nó...

Bao gồm giá nhà ở trung bình của hàng nghìn quận huyện, cũng như các dữ liệu khác.

Với tất cả những thông tin này, giờ đây bạn đã sẵn sàng bắt đầu thiết kế hệ thống của mình. Trước tiên, hãy xác định loại hình huấn luyện mà mô hình cần: có giám sát, không giám sát, bán giám sát, tự giám sát hay học tăng cường? Và đó là nhiệm vụ phân loại, hồi quy hay loại nhiệm vụ nào khác? Bạn nên sử dụng kỹ thuật học theo lô hay học trực tuyến? Trước khi đọc tiếp, hãy tạm dừng và thử tự trả lời những câu hỏi này.

Bạn đã tìm ra câu trả lời chưa? Hãy xem nào: rõ ràng đây là một bài toán học có giám sát điển hình, vì mô hình có thể được huấn luyện bằng các ví dụ được gán nhãn (mỗi trường hợp đi kèm với đầu ra dự kiến, tức là giá nhà trung bình của quận). Đây cũng là một bài toán hồi quy điển hình, vì mô hình sẽ được yêu cầu dự đoán một giá trị. Cụ thể hơn, đây là một bài toán hồi quy đa biến, vì hệ thống sẽ sử dụng nhiều đặc trưng để đưa ra dự đoán (nó sẽ sử dụng dân số của quận, thu nhập trung bình, v.v.). Đây cũng là một bài toán hồi quy đơn biến, vì chúng ta chỉ đang cố gắng dự đoán một giá trị duy nhất cho mỗi quận. Nếu chúng ta đang cố gắng dự đoán nhiều giá trị cho mỗi quận, đó sẽ là một bài toán hồi quy đa biến. Cuối cùng, không có luồng dữ liệu liên tục đổ vào hệ thống, không cần thiết phải điều chỉnh nhanh chóng theo dữ liệu thay đổi và dữ liệu đủ nhỏ để nằm gọn trong bộ nhớ, vì vậy học theo lô thông thường sẽ hoạt động tốt.

MẸO

Nếu dữ liệu quá lớn, bạn có thể chia nhỏ công việc học theo lô của mình ra nhiều máy chủ (sử dụng kỹ thuật MapReduce) hoặc sử dụng kỹ thuật học trực tuyến.

Chọn một thước đo hiệu suất

Bước tiếp theo của bạn là chọn một thước đo hiệu suất. Một thước đo hiệu suất điển hình cho các bài toán hồi quy là Sai số bình phương trung bình gốc (RMSE). Nó cho biết hệ thống thường mắc bao nhiêu lỗi trong quá trình hồi quy.

các dự đoán, với trọng số cao hơn được dành cho các lỗi lớn. **Phương trình 2-1** thể hiện công thức toán học để tính RMSE.

Phương trình 2-1. Sai số bình phương trung bình gốc (RMSE)

$$\text{RMSE}(X, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(x(i)) - y(i))^2}$$

KÝ HIỆU

Phương trình này giới thiệu một số ký hiệu rất phổ biến trong Học máy mà tôi sẽ sử dụng xuyên suốt cuốn sách này:

- m là số lượng mẫu trong tập dữ liệu mà bạn đang dùng để đo RMSE.
- Ví dụ, nếu bạn đang đánh giá RMSE trên một tập dữ liệu kiểm định gồm 2.000 quận, thì $m = 2.000$.
- (i) x là một vectơ chứa tất cả các giá trị đặc trưng (không bao gồm nhãn) của trường hợp (i) i trong tập dữ liệu, và y là nhãn của nó (giá trị đầu ra mong muốn cho trường hợp đó).
- Ví dụ, nếu quận đầu tiên trong tập dữ liệu nằm ở kinh độ $-118,29^\circ$, vĩ độ $33,91^\circ$, có 1.416 cư dân với thu nhập trung vị là 38.372 đô la và giá trị nhà trung vị là 156.400 đô la (tạm thời bỏ qua các đặc điểm khác), thì:

$$x^{(1)} = \begin{bmatrix} -118,29 \\ 33,91 \\ 1.416 \\ 38.372 \end{bmatrix}$$

Và:

$$y^{(1)} = 156.400 \text{ năm}$$

- X là một ma trận chứa tất cả các giá trị đặc trưng (không bao gồm nhãn) của tất cả các trường hợp trong tập dữ liệu. Có một hàng cho mỗi trường hợp và (i) i Hàng i bằng ma trận chuyển vị của $x^{(i)}$, ký hiệu $(x^{(i)})^T$.
- Ví dụ, nếu quận đầu tiên được mô tả như trên, thì ma trận X sẽ trông như thế này:

$$X = \begin{pmatrix} (x(1)) \\ (x(2)) \\ \vdots \\ (x(1999)) \\ (x(2000)) \end{pmatrix} = \begin{pmatrix} 118,29 & 33,91 & 1.416 & 38.372 \end{pmatrix}$$

- h là hàm dự đoán của hệ thống, còn được gọi là giả thuyết. (i) Khi hệ thống của bạn được cung cấp vectơ đặc trưng x của một trường hợp (i), nó sẽ xuất ra giá trị dự đoán $\hat{y} = h(x)$ cho trường hợp đó ($\hat{y}$ được phát âm là "y-hat").
- Ví dụ, nếu hệ thống của bạn dự đoán rằng giá nhà ở trung bình (1) ở quận đầu tiên là 158.400 đô la, thì $\hat{y} = h(x) = (1) 158.400$. Sai số dự đoán cho khu vực này là $\hat{y} - y = 2.000$.
- $RMSE(X, h)$ là hàm chi phí được đo trên tập hợp các ví dụ sử dụng giả thuyết h của bạn.

Chúng tôi sử dụng chữ thường in nghiêng cho các giá trị vô hướng (như m hoặc y) và tên hàm (như h), chữ thường in đậm cho các vectơ (như (i) x) và chữ in hoa in đậm cho các ma trận (như X).

Mặc dù RMSE thường là thước đo hiệu suất được ưa chuộng cho các tác vụ hồi quy, nhưng trong một số trường hợp, bạn có thể muốn sử dụng một hàm khác. Ví dụ, giả sử có nhiều quận/huyện ngoại lệ. Trong trường hợp đó, bạn có thể xem xét sử dụng sai số tuyệt đối trung bình (MAE, còn được gọi là độ lệch tuyệt đối trung bình; xem [Phương trình 2-2](#)):

Phương trình 2-2. Sai số tuyệt đối trung bình (MAE)

$$MAE(X, h) = \frac{1}{m} \sum_{i=1}^m |y - h(x(i))|$$

Cả RMSE và MAE đều là những cách để đo khoảng cách giữa hai vectơ: vectơ dự đoán và vectơ giá trị mục tiêu.

Có thể áp dụng nhiều biện pháp hoặc quy chuẩn khác nhau:

- Tính căn bậc hai của tổng bình phương (RMSE) tương ứng với chuẩn Euclidean: đây là khái niệm về khoảng cách mà bạn đã quen thuộc. Nó cũng được gọi là chuẩn ℓ_2 , ký hiệu là ℓ_2 hoặc chỉ ℓ (nếu không có gì khác). ℓ_2
- Tính tổng các giá trị tuyệt đối (MAE) tương ứng với chuẩn ℓ_1 , được ký hiệu là ℓ_1 . Điều này đôi khi được gọi là chuẩn Manhattan vì nó đo khoảng cách giữa hai điểm trong thành phố nếu bạn chỉ có thể di chuyển dọc theo các dãy nhà vuông góc với nhau.
- Nói một cách tổng quát hơn, chuẩn ℓ_k của một vectơ v chứa n phần tử được định nghĩa là $\|v\|_k = (\sum_{i=1}^n |v_i|^k)^{1/k}$. ℓ_k biểu thị số lượng phần tử khác 0 trong vectơ, và ℓ_∞ biểu thị giá trị tuyệt đối lớn nhất trong vectơ.
- Chỉ số chuẩn càng cao, nó càng tập trung vào các giá trị lớn và bỏ qua các giá trị nhỏ. Đó là lý do tại sao RMSE nhạy cảm với các giá trị ngoại lệ hơn MAE. Nhưng khi các giá trị ngoại lệ hiếm gặp theo cấp số mũ (như trong đường cong hình chuông), RMSE hoạt động rất tốt và thường được ưu tiên hơn.

Kiểm tra các giả định

Cuối cùng, việc liệt kê và kiểm tra lại các giả định đã được đưa ra (bởi bạn hoặc người khác) là một việc làm tốt; điều này có thể giúp bạn phát hiện ra các vấn đề nghiêm trọng từ sớm. Ví dụ, giá cả khu vực mà hệ thống của bạn xuất ra sẽ được đưa vào một hệ thống Học máy ở phía sau, và bạn giả định rằng những giá này sẽ được sử dụng như vậy. Nhưng điều gì sẽ xảy ra nếu hệ thống ở phía sau chuyển đổi giá thành các danh mục (ví dụ: “rẻ”, “trung bình” hoặc “đắt”) và sau đó sử dụng các danh mục đó thay vì chính giá cả? Trong trường hợp này, việc có được giá chính xác tuyệt đối không quan trọng; hệ thống của bạn chỉ cần xác định đúng danh mục. Nếu vậy, thì vấn đề nên được đặt ra như một bài toán phân loại, chứ không phải là bài toán hồi quy.

Nhiệm vụ. Bạn chắc chắn không muốn phát hiện ra điều này sau khi đã dành hàng tháng trời làm việc với hệ thống kiểm thử hồi quy.

May mắn thay, sau khi trao đổi với nhóm phụ trách hệ thống phía hạ nguồn, bạn tin chắc rằng họ thực sự cần giá cả thực tế, chứ không chỉ là danh mục. Tuyệt vời! Mọi thứ đã sẵn sàng, đèn xanh bật sáng, và bạn có thể bắt đầu lập trình ngay bây giờ!

Lấy dữ liệu

Đã đến lúc bắt tay vào thực hành. Đừng ngần ngại mở máy tính xách tay và xem qua các ví dụ mã. Như tôi đã đề cập trong lời tựa, tất cả các ví dụ mã trong cuốn sách này đều là mã nguồn mở và có sẵn trực tuyến tại <https://github.com/ageron/handson-ml3>, Chúng ta đang sử dụng Jupyter Notebook, là các tài liệu tương tác chứa văn bản, hình ảnh và các đoạn mã có thể thực thi (trong trường hợp này là Python). Trong cuốn sách này, tôi giả định bạn đang chạy các notebook này trên Google Colab, một dịch vụ miễn phí cho phép bạn chạy bất kỳ Jupyter Notebook nào trực tiếp trực tuyến mà không cần cài đặt bất cứ thứ gì trên máy tính của mình. Nhưng nếu bạn muốn sử dụng một nền tảng trực tuyến khác (ví dụ: Kaggle kernels) hoặc nếu bạn muốn cài đặt mọi thứ cục bộ trên máy tính của mình, vui lòng xem hướng dẫn trên trang chủ của dự án này tại trang GitHub (ở trên).

Chạy các ví dụ mã bằng Google Colab

Đầu tiên, hãy mở trình duyệt web và truy cập <https://homl.info/colab3>: Thao tác này sẽ dẫn bạn đến Google Colab, và nó sẽ hiển thị danh sách các sổ tay Jupyter cho cuốn sách này (xem [Hình 2-3](#)). Bạn sẽ tìm thấy một sổ tay cho mỗi chương, cộng thêm một vài sổ tay bổ sung, bao gồm một chương bổ sung về Máy hỗ trợ vectơ (Support Vector Machines), và các bài hướng dẫn về NumPy, Matplotlib, Pandas, Đại số tuyến tính và Vi phân.

Examples

Recent

Google Drive

GitHub

Upload

Enter a GitHub URL or search by organization or user

☐ Include private repos

ageron

Q

Repository: [🔗](#)

Branch: [🔗](#)

ageron/handson-ml3 ▼

master ▼

Path

 01_the_machine_learning_landscape.ipynb

 02_end_to_end_machine_learning_project.ipynb

 03_classification.ipynb

 04_training_linear_models.ipynb

Hình 2-3. Danh sách các sổ tay trong Google Colab

Ví dụ, nếu bạn nhấp vào

`02_end_to_end_machine_learning_project.ipynb`, sổ tay từ **Chương 2** sẽ mở ra trong Google Colab (xem **Hình 2-4**).

02_end_to_end_machine_learning_project.ipynb Share

File Edit View Insert Runtime Tools Help

+ Code + Text Copy to Drive RAM Disk Editing

Chapter 2 - End-to-end Machine Learning Project

Welcome to Machine Learning Housing Corp.! Your task is to predict median house values in Californian districts, given a number of features from these districts.

This notebook contains all the sample code and solutions to the exercises in chapter 2.

[Open in Colab](#) [Open in Kaggle](#)

Setup

First, let's import a few common modules, ensure Matplotlib plots figures inline and prepare a function to save the figures.

```
[ ] # Python ≥3.8 is required
import sys
assert sys.version_info >= (3, 8)
```

Text cells:
double-click to edit

Code cells:
click to edit

Hình 2-4. Sổ tay của bạn trong Google Colab

Một Jupyter notebook bao gồm một danh sách các ô. Mỗi ô chứa mã thực thi hoặc văn bản. Hãy thử nhấp đúp vào ô văn bản đầu tiên (chứa câu "Chào mừng đến với Tập đoàn Nhà ở Học máy!").

Thao tác này sẽ mở ô để chỉnh sửa. Lưu ý rằng sổ tay Jupyter sử dụng cú pháp Markdown để định dạng (ví dụ: ****in đậm****, **in nghiêng**, # Tiêu đề, [url](văn bản liên kết), v.v.). Hãy thử sửa đổi văn bản này, sau đó nhấn Shift-Enter để xem kết quả.

Tiếp theo, tạo một ô mã mới bằng cách chọn Chèn > Ô mã từ menu.

Ngoài ra, bạn có thể nhấp vào nút + Code trên thanh công cụ, hoặc di chuột đến cuối ô cho đến khi thấy + Code và + Text xuất hiện, sau đó nhấp vào + Code. Trong ô mã mới, hãy nhập một số mã Python, chẳng hạn như `print("Hello World")`, sau đó nhấn Shift-Enter để chạy mã này (hoặc nhấp vào nút ở bên trái ô).

Nếu bạn chưa đăng nhập vào tài khoản Google, bạn sẽ được yêu cầu đăng nhập ngay bây giờ (nếu bạn chưa có tài khoản Google, bạn cần tạo một tài khoản). Sau khi đăng nhập và bạn thử chạy mã, bạn sẽ nhận được cảnh báo bảo mật cho biết rằng sổ tay này không phải do Google tạo ra.

Thực tế, một kẻ xấu có thể tạo ra một sổ tay nhằm lừa bạn nhập thông tin đăng nhập Google để chúng có thể truy cập dữ liệu cá nhân của bạn.

Vì vậy, trước khi chạy một notebook, hãy luôn đảm bảo bạn tin tưởng tác giả của nó (hoặc kiểm tra kỹ chức năng của từng ô mã trước khi chạy). Bây giờ, giả sử bạn tin tưởng tôi (hoặc bạn dự định kiểm tra từng ô mã), bạn có thể nhấn vào "Chạy dù sao đi nữa".

Colab sẽ cấp phát cho bạn một Runtime mới : đây là một máy ảo miễn phí nằm trên máy chủ của Google, chứa nhiều công cụ và thư viện Python, bao gồm mọi thứ chúng ta cần cho hầu hết các chương (trong một số chương, chúng ta sẽ cần chạy một lệnh để cài đặt thêm một số thư viện). Quá trình này sẽ mất vài giây. Tiếp theo, Colab sẽ tự động kết nối với Runtime này và sử dụng nó để thực thi ô mã mới của bạn.

Điều quan trọng cần lưu ý là mã này chạy trên môi trường Runtime, chứ không phải trên máy tính của bạn. Mã này...

Kết quả sẽ được hiển thị bên dưới ô. Chúc mừng, bạn đã chạy thành công mã Python trên Colab!

MẸO

Để chèn một ô mã mới, bạn cũng có thể nhấn Ctrl-m (hoặc Cmd-m trên macOS) theo sau là a (để chèn phía trên ô đang hoạt động) hoặc b (để chèn phía dưới). Có rất nhiều phím tắt khác: bạn có thể xem và chỉnh sửa chúng bằng cách nhấn Ctrl-m (hoặc Cmd-m) rồi h. Nếu bạn chọn chạy các notebook trên Kaggle hoặc trên máy tính của mình bằng JupyterLab hoặc một IDE như Visual Studio Code với tiện ích mở rộng Jupyter, bạn sẽ thấy một số khác biệt nhỏ: Môi trường chạy được gọi là Kernel, giao diện người dùng và các phím tắt hơi khác nhau, v.v. Nhưng việc chuyển đổi từ môi trường Jupyter này sang môi trường Jupyter khác không quá khó.

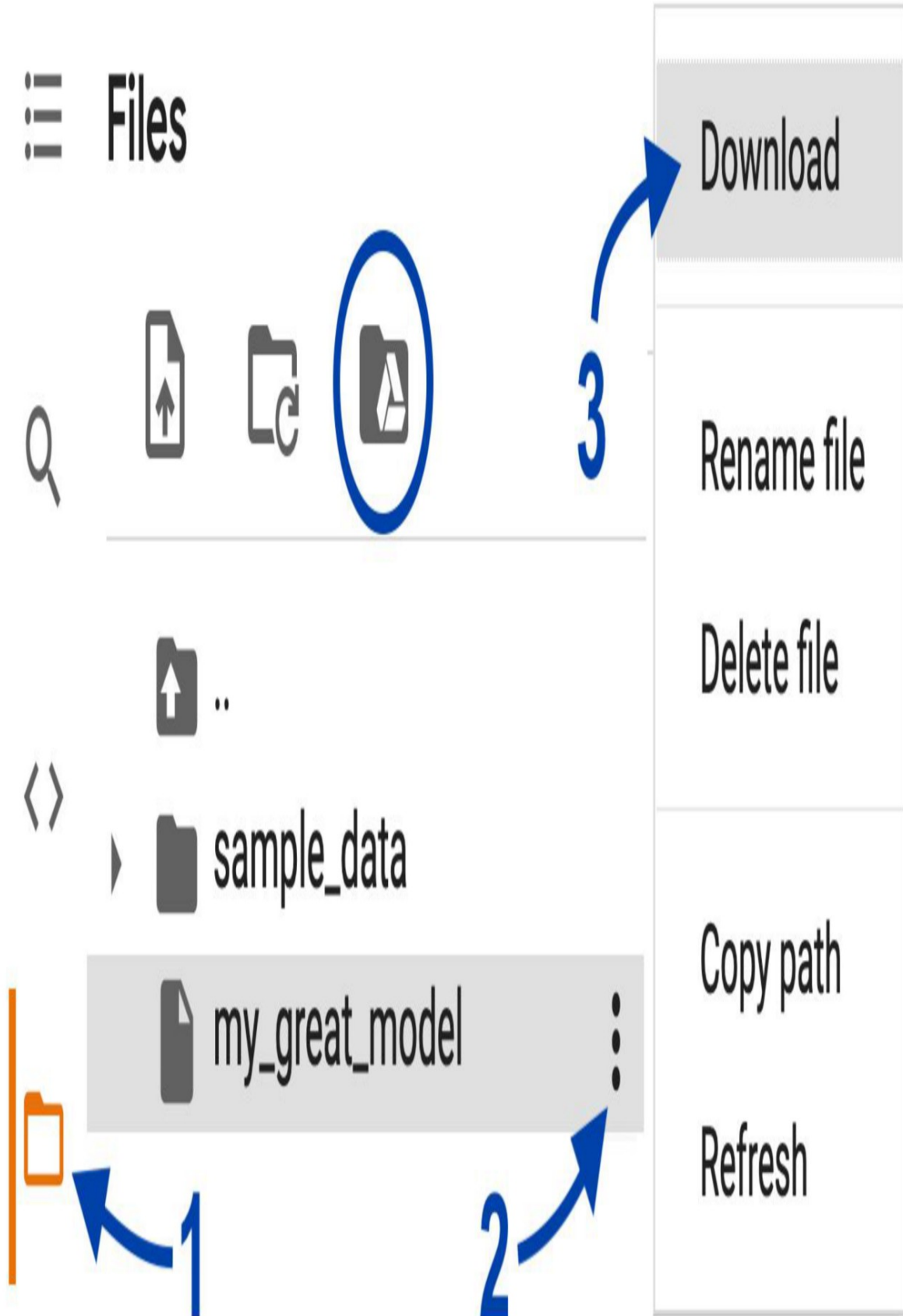
Lưu các thay đổi mã và dữ liệu của bạn

Bạn có thể thực hiện thay đổi trên sổ tay Colab, và những thay đổi đó sẽ được lưu giữ miễn là bạn vẫn mở tab trình duyệt, nhưng một khi bạn đóng tab, các thay đổi sẽ bị mất. Để tránh điều này, hãy đảm bảo bạn lưu một bản sao của sổ tay vào Google Drive bằng cách chọn Tệp > Lưu bản sao vào Drive. Hoặc, bạn có thể tải sổ tay xuống máy tính bằng cách chọn Tệp > Tải xuống > Tải xuống .ipynb. Bằng cả hai cách, bạn đều có thể truy cập <https://colab.research.google.com/> sau đó. và mở lại sổ tay (từ Google Drive hoặc bằng cách tải lên từ máy tính của bạn).

CẢNH BÁO

Google Colab chỉ được thiết kế để sử dụng tương tác: bạn có thể thử nghiệm trong các sổ tay và chỉnh sửa mã theo ý muốn, nhưng bạn không thể để các sổ tay chạy tự động trong thời gian dài, nếu không Runtime sẽ bị tắt và tất cả dữ liệu sẽ bị mất.

Nếu sổ tay tạo ra dữ liệu mà bạn quan tâm, hãy đảm bảo bạn tải xuống dữ liệu này trước khi Runtime tắt. Để làm điều này, hãy nhấp vào biểu tượng Tệp (xem bước 1 trong [Hình 2-5](#)), tìm tệp bạn muốn tải xuống, nhấp vào dấu chấm dọc bên cạnh tệp đó (bước 2) và nhấp vào Tải xuống (bước 3).



Hình 2-5. Tải xuống tệp từ Google Colab Runtime (các bước 1 đến 3) hoặc gắn kết Google Drive của bạn (biểu tượng khoanh tròn)

Ngoài ra, bạn cũng có thể gắn kết Google Drive của mình vào Runtime, cho phép sổ tay đọc và ghi trực tiếp các tệp vào Google Drive, như thể đó là một thư mục cục bộ. Để làm điều này, hãy nhấp vào biểu tượng Tệp (bước 1), sau đó nhấp vào biểu tượng Google Drive (được khoanh tròn trong Hình 2-5) và làm theo hướng dẫn trên màn hình. Theo mặc định, Google Drive của bạn sẽ được gắn kết tại `/content/drive/MyDrive`. Nếu bạn muốn sao lưu một tệp dữ liệu, chỉ cần sao chép nó vào thư mục này bằng cách chạy lệnh `!cp /content/my_great_model /content/drive/MyDrive`. Bất kỳ lệnh nào bắt đầu bằng dấu chấm than (!) đều được coi là lệnh shell, không phải là mã Python: `cp` là lệnh shell của Linux để sao chép tệp từ đường dẫn này sang đường dẫn khác. Lưu ý rằng Colab Runtime chạy trên Linux (cụ thể là Ubuntu).

Sức mạnh và mối nguy hiểm của tính tương tác

Jupyter notebook có tính tương tác cao, và đó là một điều tuyệt vời: bạn có thể chạy từng ô mã một, dừng lại ở bất kỳ điểm nào, chèn thêm một ô mã khác, thử nghiệm với mã, quay lại và chạy lại cùng một ô mã đó, v.v., và tôi rất khuyến khích bạn làm như vậy. Nếu bạn chỉ chạy các ô mã một cách riêng lẻ mà không bao giờ thử nghiệm với chúng, bạn sẽ không học nhanh được. Tuy nhiên, tính linh hoạt này cũng có cái giá của nó: rất dễ chạy các ô mã sai thứ tự, hoặc quên chạy một ô mã nào đó. Nếu điều này xảy ra, các ô mã tiếp theo có thể sẽ bị lỗi. Ví dụ, ô mã đầu tiên trong mỗi notebook chứa mã thiết lập (chẳng hạn như các lệnh `import`), vì vậy hãy đảm bảo bạn chạy nó trước, nếu không thì mọi thứ sẽ không hoạt động.

MẸO

Nếu bạn gặp phải lỗi lạ, hãy thử khởi động lại Runtime (bằng cách chọn Runtime > Restart runtime từ menu) và sau đó chạy lại tất cả các ô từ đầu sổ tay: điều này thường sẽ giải quyết được vấn đề. Nếu không, thì có khả năng một trong những thay đổi bạn đã thực hiện đã làm hỏng sổ tay: chỉ cần quay lại sổ tay ban đầu và thử lại.
Nếu vẫn không được, vui lòng báo cáo sự cố trên GitHub.

Mã sách so với mã sổ tay

Đôi khi bạn có thể nhận thấy một vài khác biệt nhỏ giữa mã trong cuốn sách này và mã trong các tập tài liệu tham khảo. Điều này có thể xảy ra vì một số lý do:

- Có thể thư viện đã thay đổi đôi chút vào thời điểm bạn đọc những dòng này, hoặc có lẽ dù đã cố gắng hết sức, tôi vẫn mắc lỗi trong cuốn sách. Rất tiếc, tôi không thể sửa lỗi mã trong bản sao cuốn sách của bạn một cách thần kỳ (trừ khi bạn đang đọc bản điện tử và có thể tải xuống phiên bản mới nhất), nhưng tôi có thể sửa các notebook: vì vậy, nếu bạn gặp lỗi sau khi sao chép mã từ cuốn sách này, vui lòng tìm mã đã được sửa trong các notebook: tôi sẽ cố gắng giữ cho chúng không có lỗi và được cập nhật với các phiên bản thư viện mới nhất.
- Các tập tin này chứa một số đoạn mã bổ sung để làm đẹp hình ảnh (thêm nhãn, thiết lập kích thước phông chữ, v.v.) và lưu chúng ở độ phân giải cao cho cuốn sách này. Bạn có thể bỏ qua đoạn mã bổ sung này nếu muốn.

Tôi đã tối ưu hóa mã nguồn để dễ đọc và đơn giản hơn: tôi đã làm cho nó càng tuyến tính và phẳng càng tốt, chỉ định rất ít hàm hoặc lớp. Mục tiêu là để đảm bảo rằng mã bạn đang chạy thường nằm ngay trước mắt bạn, chứ không bị lồng ghép trong nhiều lớp trừu tượng mà bạn phải tìm kiếm.

Điều này cũng giúp bạn dễ dàng làm việc với mã nguồn hơn. Để đơn giản, việc xử lý lỗi được hạn chế, và tôi đã đặt một số lệnh import ít phổ biến nhất ngay tại những nơi cần thiết (thay vì đặt chúng ở đầu tệp, như khuyến nghị của hướng dẫn kiểu Python PEP 8). Tuy nhiên, mã nguồn sản xuất của bạn sẽ không khác biệt nhiều: chỉ là có tính mô-đun hơn một chút, và thêm các bài kiểm tra và xử lý lỗi, thế thôi.

Được rồi! Khi bạn đã quen thuộc với Colab, bạn đã sẵn sàng tải xuống dữ liệu.

Tải xuống dữ liệu

Trong môi trường thông thường, dữ liệu của bạn sẽ được lưu trữ trong cơ sở dữ liệu quan hệ hoặc một kho lưu trữ dữ liệu phổ biến khác, và được phân tán trên nhiều hệ thống khác nhau.

Bảng/tài liệu/tệp tin. Để truy cập, trước tiên bạn cần lấy thông tin đăng nhập và quyền truy cập, đồng thời làm quen với cấu trúc dữ liệu⁴. Tuy nhiên, trong dự án này, mọi thứ đơn giản hơn nhiều: bạn chỉ cần tải xuống một tệp nén duy nhất, `housing.tgz`, chứa một tệp giá trị phân tách bằng dấu phẩy (CSV) có tên `housing.csv` với tất cả dữ liệu.

Thay vì tải xuống và giải nén dữ liệu thủ công, thường thì viết một hàm tự động thực hiện việc này sẽ tốt hơn. Điều này đặc biệt hữu ích nếu dữ liệu thay đổi thường xuyên: bạn có thể viết một đoạn mã nhỏ sử dụng hàm đó để lấy dữ liệu mới nhất (hoặc bạn có thể thiết lập một tác vụ theo lịch trình để thực hiện việc này tự động theo định kỳ). Tự động hóa quá trình lấy dữ liệu cũng hữu ích nếu bạn cần cài đặt tập dữ liệu trên nhiều máy tính.

Đây là hàm dùng để lấy và tải dữ liệu:

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")

    return pd.read_csv(Path("datasets/housing/housing.csv"))

nhà ở = tải dữ liệu nhà ở()
```

Khi hàm `load_housing_data()` được gọi, nó sẽ tìm kiếm tệp `datasets/housing.tgz`, và nếu không tìm thấy, nó sẽ tạo thư mục `datasets` bên trong thư mục hiện tại (mặc định là `/content` trong Colab), tải xuống tệp `housing.tgz` từ kho lưu trữ GitHub `ageron/data`, giải nén nội dung của tệp vào thư mục `datasets`, và điều này tạo ra thư mục `datasets/housing` với tệp `housing.csv` bên trong. Cuối cùng,

Hàm này tải tệp CSV này vào một đối tượng Pandas DataFrame chứa tất cả dữ liệu và trả về đối tượng đó.

Hãy cùng xem qua cấu trúc dữ liệu.

Hãy cùng xem năm hàng dữ liệu đầu tiên bằng cách sử dụng phương thức `head()` của DataFrame (xem [Hình 2-6](#)).

```
housing.head()
```

	longitude	latitude	housing_median_age	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41.0	8.3252	NEAR BAY	452600.0
1	-122.22	37.86	21.0	8.3014	NEAR BAY	358500.0
2	-122.24	37.85	52.0	7.2574	NEAR BAY	352100.0
3	-122.25	37.85	52.0	5.6431	NEAR BAY	341300.0
4	-122.25	37.85	52.0	3.8462	NEAR BAY	342200.0

Hình 2-6. Năm hàng đầu tiên trong tập dữ liệu

Mỗi hàng đại diện cho một quận. Có 10 thuộc tính (không phải tất cả đều là thuộc tính này).
(như trong ảnh chụp màn hình): kinh độ, vĩ độ,
tuổi trung bình của nhà ở, tổng số phòng, tổng số phòng ngủ,
dân số, hộ gia đình, thu nhập trung vị,
giá trị nhà trung bình và khoảng cách đến biển.

Phương thức `info()` rất hữu ích để có được mô tả nhanh về dữ liệu, trong
cụ thể là tổng số hàng, kiểu dữ liệu của từng thuộc tính và số lượng.
các giá trị khác 0:

```
>>> housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 mục, từ 0 đến 20639
Các cột dữ liệu (tổng cộng 10 cột):
# Cột                               Số lượng không rỗng Dtype
--- -----
0 kinh độ 20640 số thực 64 không rỗng
1 vĩ độ 20640 không rỗng float64
2 housing_median_age 20640 non-null float64
3 total_rooms 20640 non-null float64
4 total_bedrooms 20433 non-null float64
5 dân số 20640 không rỗng float64
6 hộ gia đình 20640 số thực không rỗng 64
7 median_income 20640 non-null float64
8 median_house_value 20640 non-null float64
9 ocean_proximity 20640 đối tượng không rỗng
dtypes: float64(9), object(1)
Mức sử dụng bộ nhớ: 1.6+ MB
```

GHI CHÚ

Trong cuốn sách này, khi một ví dụ mã chứa cả mã lệnh và kết quả đầu ra, như trường hợp này
ở đây, nó được định dạng giống như trong trình thông dịch Python để dễ đọc hơn: các dòng mã
Các phần đầu được đánh dấu bằng `>>>` (hoặc ... đối với các khối thực tế), còn các phần đầu ra thì không có tiền tố.

Tập dữ liệu này có 20.640 trường hợp, điều đó có nghĩa là nó khá nhỏ.
Theo tiêu chuẩn của Học máy, nhưng nó hoàn hảo để bắt đầu. Hãy lưu ý rằng
Thuộc tính `total_bedrooms` chỉ có 20.433 giá trị không rỗng, có nghĩa là

Hiện có 207 quận/huyện thiếu tính năng này. Chúng ta cần phải khắc phục vấn đề này.
sau đó.

Tất cả các thuộc tính đều là số, ngoại trừ trường ocean_proximity. Kiểu dữ liệu của nó là số.
Nó là một đối tượng, vì vậy nó có thể chứa bất kỳ loại đối tượng Python nào. Nhưng vì bạn
Tôi đã tải dữ liệu này từ một tệp CSV, vì vậy bạn biết rằng đó phải là một thuộc tính văn bản.
Khi nhìn vào năm hàng đầu tiên, có lẽ bạn đã nhận thấy các giá trị
Các giá trị trong cột ocean_proximity bị lặp lại, điều đó có nghĩa là...
Có lẽ đó là một thuộc tính phân loại. Bạn có thể tìm hiểu xem những danh mục nào tồn tại và
xác định số lượng quận/huyện thuộc mỗi loại bằng cách sử dụng công thức sau:
Phương thức value_counts():

```
>>> housing["ocean_proximity"].value_counts()
<1H ĐẠI DƯƠNG      9136
NỘI ĐỊA              6551
GẦN ĐẠI DƯƠNG      2658
GẦN VỊNH             2290
ĐẢO 5                 5
Tên: ocean_proximity, kiểu dữ liệu: int64
```

Hãy xem xét các trường khác. Phương thức describe() hiển thị bản tóm tắt.
của các thuộc tính số (Hình 2-7).

```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	500001.000000

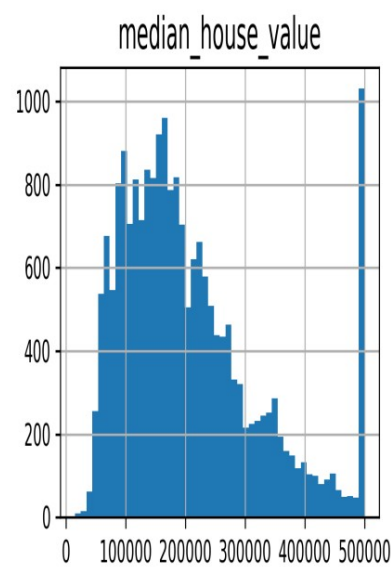
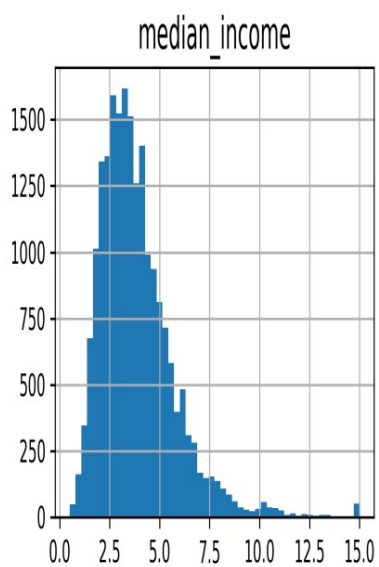
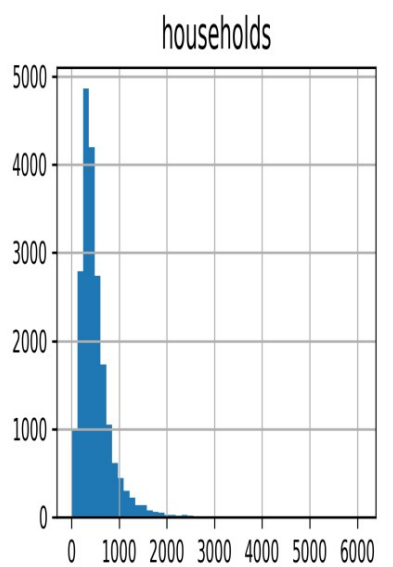
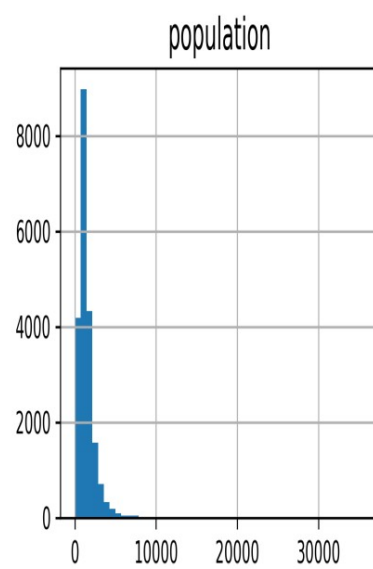
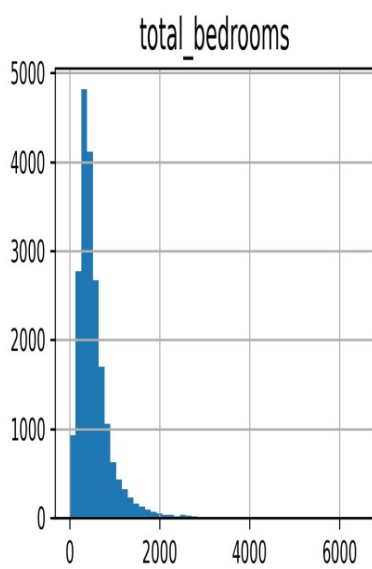
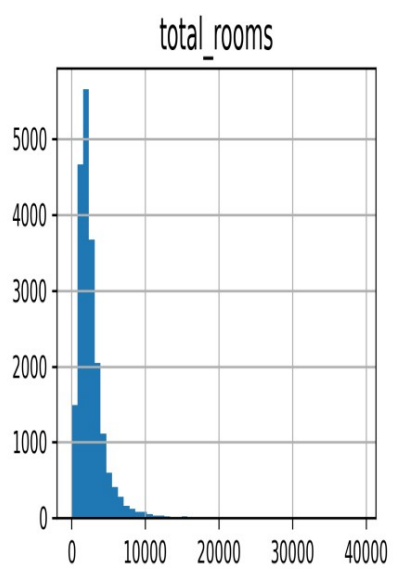
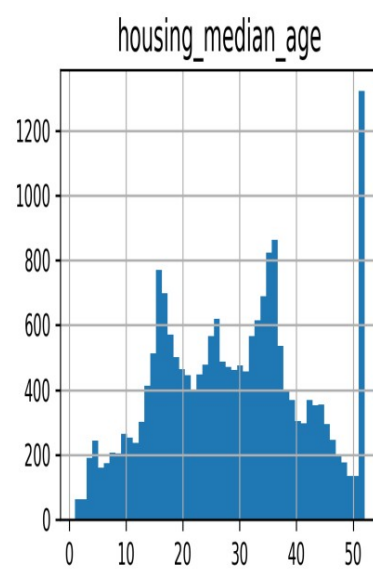
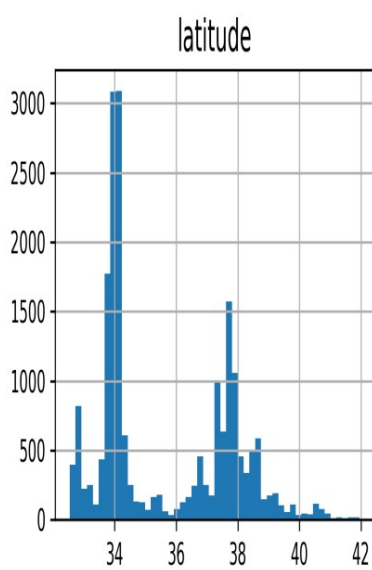
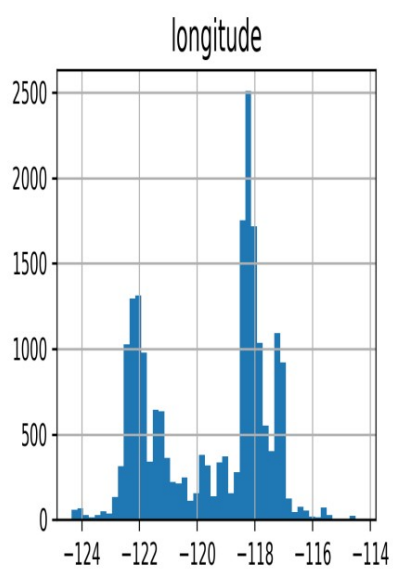
Hình 2-7. Tóm tắt từng thuộc tính số.

Các hàng đếm, trung bình, nhỏ nhất và lớn nhất khá dễ hiểu. Lưu ý rằng các giá trị rỗng sẽ bị bỏ qua (ví dụ, số lượng tổng số phòng ngủ là 20.433, chứ không phải 20.640). Hàng độ lệch chuẩn hiển thị độ lệch chuẩn, đo lường mức độ phân tán của các giá trị. Các hàng 25%, 50% và 75% hiển thị các phân vị tương ứng: phân vị cho biết giá trị mà dưới đó một tỷ lệ phần trăm nhất định các quan sát trong một nhóm quan sát nằm. Ví dụ, 25% các quận có tuổi trung vị nhà ở thấp hơn 18, trong khi 50% thấp hơn 29 và 75% thấp hơn 37. Chúng thường được gọi là phân vị thứ 25 (hoặc tứ phân vị thứ nhất), trung vị và phân vị thứ 75 (hoặc tứ phân vị thứ ba).

Một cách nhanh chóng khác để nắm bắt loại dữ liệu bạn đang xử lý là vẽ biểu đồ tần số cho từng thuộc tính số. Biểu đồ tần số hiển thị số lượng trường hợp (trục tung) có phạm vi giá trị nhất định (trục hoành). Bạn có thể vẽ biểu đồ cho từng thuộc tính một hoặc gọi phương thức `hist()` trên toàn bộ tập dữ liệu (như trong ví dụ mã sau), và nó sẽ vẽ biểu đồ tần số cho từng thuộc tính số (xem [Hình 2-8](#)):

```
import matplotlib.pyplot as plt

housing.hist(bins=50, figsize=(12, 8)) plt.show()
```



Hình 2-8. Biểu đồ tần suất cho từng thuộc tính số.

Có một vài điều bạn có thể nhận thấy trong các biểu đồ tần số này:

1. Đầu tiên, thuộc tính thu nhập trung vị dường như không được thể hiện bằng đô la Mỹ (USD). Sau khi kiểm tra với nhóm thu thập dữ liệu, bạn được cho biết rằng dữ liệu đã được điều chỉnh tỷ lệ và giới hạn ở mức 15 (thực tế là 15.0001) đối với thu nhập trung vị cao hơn, và ở mức 0.5 (thực tế là 0.4999) đối với thu nhập trung vị thấp hơn. Các con số này thể hiện xấp xỉ hàng chục nghìn đô la (ví dụ: 3 thực tế có nghĩa là khoảng 30.000 đô la). Làm việc với các thuộc tính đã được xử lý trước là điều phổ biến trong Học máy, và điều đó không nhất thiết là một vấn đề, nhưng bạn nên cố gắng hiểu cách dữ liệu được tính toán.
2. Tuổi trung bình của nhà ở và giá trị trung bình của nhà ở cũng bị giới hạn. Điều này có thể là một vấn đề nghiêm trọng vì đó là thuộc tính mục tiêu của bạn (nhãn của bạn). Các thuật toán Học máy của bạn có thể học được rằng giá không bao giờ vượt quá giới hạn đó. Bạn cần kiểm tra với nhóm khách hàng của mình (nhóm sẽ sử dụng đầu ra của hệ thống) để xem đây có phải là vấn đề hay không. Nếu họ nói với bạn rằng họ cần dự đoán chính xác ngay cả khi giá vượt quá 500.000 đô la, thì bạn có hai lựa chọn:
 - a. Thu thập các nhãn phù hợp cho các quận có nhãn đã được dán kín.
 - b. Loại bỏ những quận đó khỏi tập dữ liệu huấn luyện (và cả khỏi tập dữ liệu kiểm tra, vì hệ thống của bạn không nên bị đánh giá kém nếu nó dự đoán các giá trị vượt quá 500.000 đô la).
3. Các thuộc tính này có thang đo rất khác nhau. Chúng ta sẽ thảo luận điều này sau trong chương này, khi tìm hiểu về việc mở rộng quy mô đặc trưng.
4. Cuối cùng, nhiều biểu đồ tần số bị lệch phải: chúng kéo dài xa hơn về phía bên phải của giá trị trung vị so với phía bên trái. Điều này có thể khiến một số thuật toán Học máy khó phát hiện ra các mẫu hơn. Chúng ta sẽ thử chuyển đổi các thuộc tính này sau để có được các phân bố đối xứng và hình chuông hơn.

Hi vọng giờ đây bạn đã hiểu rõ hơn về loại dữ liệu mà mình đang xử lý.

CẢNH BÁO

Khoan đã! Trước khi xem xét dữ liệu thêm nữa, bạn cần tạo một bộ dữ liệu thử nghiệm, để riêng nó ra và tuyệt đối không được xem lại.

Tạo bộ kiểm thử

Nghe có vẻ lạ khi tự nguyện tách riêng một phần dữ liệu ở giai đoạn này.

Tóm lại, bạn chỉ mới lướt qua dữ liệu một cách nhanh chóng, và chắc chắn bạn cần tìm hiểu thêm rất nhiều về nó trước khi quyết định sử dụng thuật toán nào, phải không? Điều này đúng, nhưng bộ não của bạn là một hệ thống phát hiện mẫu tuyệt vời, điều đó cũng có nghĩa là nó rất dễ bị quá khớp: nếu bạn nhìn vào tập dữ liệu thử nghiệm, bạn có thể tình cờ phát hiện ra một số mẫu có vẻ thú vị trong dữ liệu thử nghiệm dẫn đến việc bạn chọn một loại mô hình Học máy cụ thể. Khi bạn ước tính lỗi tổng quát hóa bằng cách sử dụng tập dữ liệu thử nghiệm, ước tính của bạn sẽ quá lạc quan, và bạn sẽ khởi chạy một hệ thống không hoạt động tốt như mong đợi. Điều này được gọi là sai lệch do dò tìm dữ liệu .

Về mặt lý thuyết, việc tạo một tập dữ liệu thử nghiệm khá đơn giản: chọn ngẫu nhiên một số trường hợp, thường là 20% của tập dữ liệu (hoặc ít hơn nếu tập dữ liệu của bạn rất lớn), và để chúng sang một bên:

```
import numpy as np

def shuffle_and_split_data(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

Bạn có thể sử dụng hàm này như sau:

```
>>> train_set, test_set = shuffle_and_split_data(housing, 0.2) >>> len(train_set) 16512 >>>
len(test_set) 4128
```

Cách này có hiệu quả, nhưng không hoàn hảo: nếu bạn chạy chương trình lại, nó sẽ tạo ra một tập dữ liệu thử nghiệm khác! Theo thời gian, bạn (hoặc các thuật toán Học máy của bạn) sẽ thấy toàn bộ tập dữ liệu, điều mà bạn muốn tránh.

Một giải pháp là lưu bộ dữ liệu thử nghiệm trong lần chạy đầu tiên và sau đó tải nó trong các lần chạy tiếp theo. Một lựa chọn khác là đặt hạt giống của bộ tạo số ngẫu nhiên (ví dụ: với `np.random.seed(42)`) trước khi gọi `np.random.permutation()` để nó luôn tạo ra các chỉ số được xáo trộn giống nhau.

Tuy nhiên, cả hai giải pháp này đều sẽ không hoạt động vào lần tiếp theo bạn tải về bộ dữ liệu cập nhật. Để có sự phân chia tập huấn luyện/kiểm thử ổn định ngay cả sau khi cập nhật bộ dữ liệu, một giải pháp phổ biến là sử dụng mã định danh của mỗi trường hợp để quyết định xem nó có nên được đưa vào tập kiểm thử hay không (giả sử các trường hợp có mã định danh duy nhất và không thay đổi). Ví dụ, bạn có thể tính toán mã băm của mã định danh của mỗi trường hợp và đưa trường hợp đó vào tập kiểm thử nếu mã băm nhỏ hơn hoặc bằng 20% giá trị băm tối đa. Điều này đảm bảo rằng tập kiểm thử sẽ vẫn nhất quán trong nhiều lần chạy, ngay cả khi bạn làm mới bộ dữ liệu. Tập kiểm thử mới sẽ chứa 20% số trường hợp mới, nhưng sẽ không chứa bất kỳ trường hợp nào đã từng có trong tập huấn luyện trước đó.

Dưới đây là một cách triển khai khả thi:

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio): return
    crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column): ids =
    data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))

    trả về data.loc[~in_test_set], data.loc[in_test_set]
```

Đáng tiếc là bộ dữ liệu nhà ở không có cột định danh. Giải pháp đơn giản nhất là sử dụng chỉ số hàng làm ID:

```
housing_with_id = housing.reset_index() # thêm một cột `index`
train_set, test_set = split_data_with_id_hash(housing_with_id,
0.2, "index")
```

Nếu bạn sử dụng chỉ mục hàng làm mã định danh duy nhất, bạn cần đảm bảo rằng dữ liệu mới được thêm vào cuối tập dữ liệu và không có hàng nào bị xóa. Nếu điều này không thể thực hiện được, bạn có thể thử sử dụng các đặc điểm ổn định nhất để xây dựng mã định danh duy nhất. Ví dụ, vĩ độ và kinh độ của một quận được đảm bảo ổn định trong vài triệu năm, vì vậy bạn có thể **kết** hợp chúng thành một ID như sau:

```
housing_with_id["id"] = housing["longitude"] * 1000+
housing["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id,
0.2, "id")
```

Scikit-Learn cung cấp một vài hàm để chia tập dữ liệu thành nhiều tập con theo nhiều cách khác nhau. Hàm đơn giản nhất là `train_test_split()`, về cơ bản thực hiện điều tương tự như hàm `shuffle_and_split_data()` mà chúng ta đã định nghĩa trước đó, với một vài tính năng bổ sung. Thứ nhất, có tham số `random_state` cho phép bạn thiết lập hạt giống của bộ tạo số ngẫu nhiên. Thứ hai, bạn có thể truyền cho nó nhiều tập dữ liệu có cùng số hàng, và nó sẽ chia chúng dựa trên cùng các chỉ mục (điều này rất hữu ích, ví dụ, nếu bạn có một DataFrame riêng cho nhãn):

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2,
random_state=42)
```

Cho đến nay, chúng ta đã xem xét các phương pháp lấy mẫu ngẫu nhiên thuần túy. Điều này nhìn chung là ổn nếu tập dữ liệu của bạn đủ lớn (đặc biệt là so với số lượng thuộc tính), nhưng nếu không, bạn có nguy cơ đưa vào một sai số nhất định.

Sai lệch lấy mẫu đáng kể. Khi một công ty khảo sát quyết định gọi điện cho 1.000 người để hỏi họ một vài câu hỏi, họ không chỉ chọn ngẫu nhiên 1.000 người trong danh bạ điện thoại. Họ cố gắng đảm bảo rằng 1.000 người này đại diện cho toàn bộ dân số, liên quan đến các câu hỏi mà họ muốn hỏi. Ví dụ, dân số Hoa Kỳ gồm 51,1% nữ và 48,9% nam, vì vậy một cuộc khảo sát được thực hiện tốt ở Hoa Kỳ sẽ cố gắng duy trì tỷ lệ này trong mẫu: 511 nữ và 489 nam (ít nhất là nếu chúng ta tin rằng câu trả lời có thể khác nhau giữa các giới tính). Điều này được gọi là lấy mẫu phân tầng: dân số được chia thành các nhóm nhỏ đồng nhất gọi là tầng, và số lượng mẫu phù hợp được lấy từ mỗi tầng để đảm bảo rằng tập dữ liệu thử nghiệm đại diện cho toàn bộ dân số. Nếu những người thực hiện khảo sát sử dụng phương pháp lấy mẫu ngẫu nhiên hoàn toàn, sẽ có khoảng 10,7% khả năng lấy mẫu một tập dữ liệu thử nghiệm bị lệch với tỷ lệ nữ dưới 48,5% hoặc trên 53,5%. Dù thế nào đi nữa, kết quả khảo sát rất có thể sẽ bị sai lệch đáng kể.

Giả sử bạn đã trò chuyện với các chuyên gia và họ nói rằng thu nhập trung vị là một thuộc tính rất quan trọng để dự đoán giá nhà trung vị. Bạn có thể muốn đảm bảo rằng tập dữ liệu thử nghiệm đại diện cho các nhóm thu nhập khác nhau trong toàn bộ tập dữ liệu. Vì thu nhập trung vị là một thuộc tính số liên tục, trước tiên bạn cần tạo một thuộc tính nhóm thu nhập. Hãy xem xét kỹ hơn biểu đồ phân bố thu nhập trung vị (trở lại [Hình 2-8](#)): hầu hết các giá trị thu nhập trung vị tập trung xung quanh mức 1,5 đến 6 (tức là 15.000-60.000 đô la), nhưng một số thu nhập trung vị vượt xa mức 6. Điều quan trọng là phải có đủ số lượng mẫu trong tập dữ liệu của bạn cho mỗi nhóm, nếu không ước tính về tầm quan trọng của nhóm đó có thể bị sai lệch. Điều này có nghĩa là bạn không nên có quá nhiều nhóm, và mỗi nhóm phải đủ lớn.

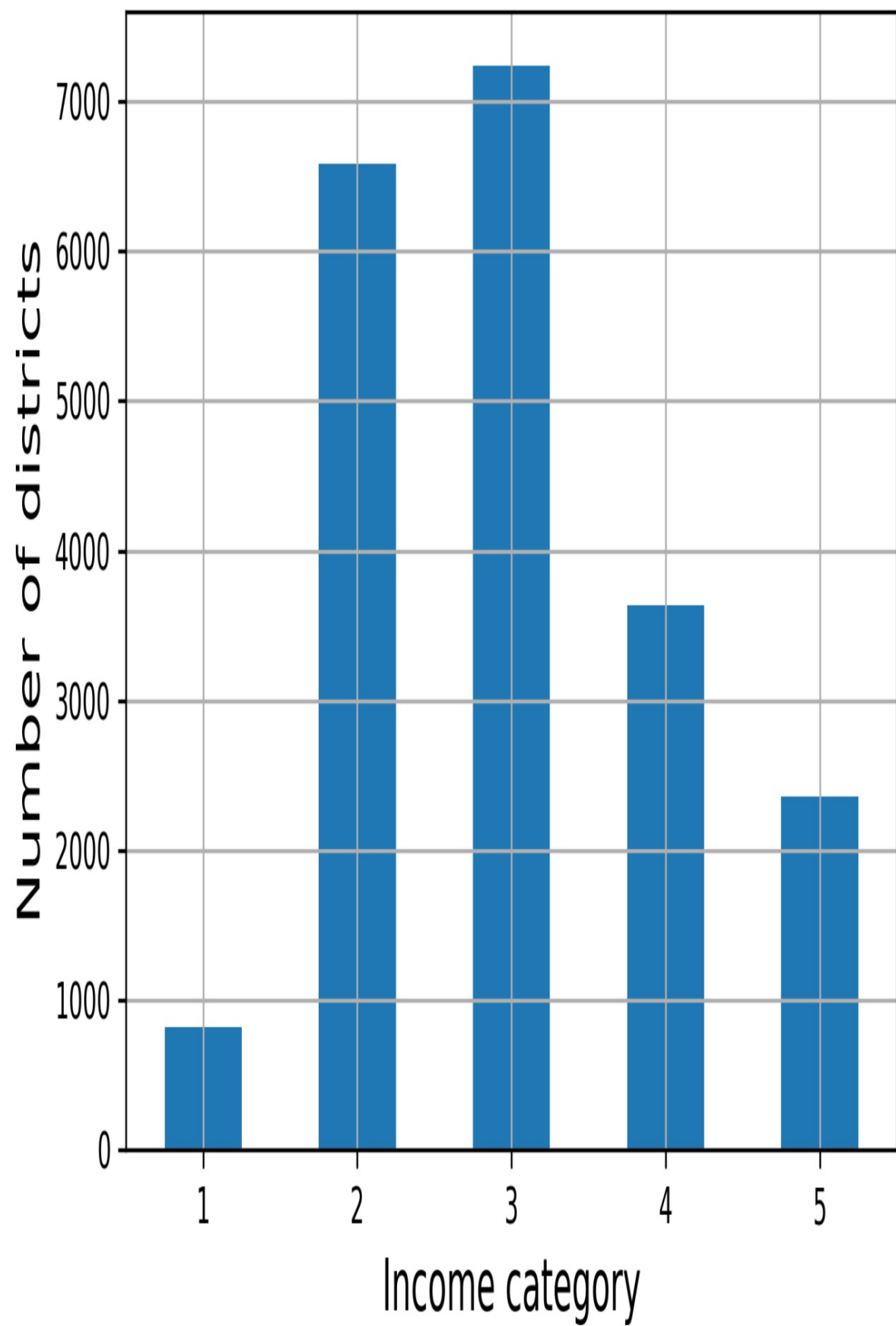
Đoạn mã sau sử dụng hàm `pd.cut()` để tạo thuộc tính danh mục thu nhập với năm danh mục (được đánh số từ 1 đến 5): danh mục 1 nằm trong khoảng từ 0 đến 1,5 (tức là dưới 15.000 đô la), danh mục 2 từ 1,5 đến 3, và

sớm:

```
housing["income_cat"] = pd.cut(housing["median_income"],
                                bins=[0., 1.5, 3.0, 4.5, 6.,
np.inf],
                                nhãn=[1, 2, 3, 4, 5])
```

Các nhóm thu nhập này được thể hiện trong **Hình 2-9**:

```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0, grid=True)
plt.xlabel("
Nhóm thu nhập") plt.ylabel("Số lượng
quận huyện") plt.show()
```



Hình 2-9. Biểu đồ tần suất các loại thu nhập

Giờ bạn đã sẵn sàng thực hiện lấy mẫu phân tầng dựa trên nhóm thu nhập.

Scikit-Learn cung cấp một số lớp splitter trong gói `sklearn.model_selection`, thực hiện nhiều chiến lược khác nhau để chia tập dữ liệu của bạn thành tập huấn luyện và tập kiểm tra. Mỗi splitter đều có phương thức `split()` trả về một iterator trên các tập dữ liệu được chia thành tập huấn luyện/kiểm tra khác nhau. Chính xác hơn, phương thức `split()` trả về chỉ số của tập huấn luyện và tập kiểm tra, chứ không phải chính dữ liệu. Việc có nhiều lần chia có thể hữu ích nếu bạn muốn ước lượng hiệu suất của mô hình tốt hơn, như chúng ta sẽ thấy khi thảo luận về kiểm định chéo sau trong chương này. Ví dụ, đoạn mã sau tạo ra 10 lần chia phân tầng khác nhau của cùng một tập dữ liệu:

```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2,
                                  random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing,
                                              housing["income_cat"]):
    strat_train_set_n = housing.loc[train_index]
    strat_test_set_n = housing.loc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

Hiện tại, chúng ta chỉ cần sử dụng phương án chia tách đầu tiên:

```
strat_train_set, strat_test_set = strat_splits[0]
```

Vì lấy mẫu phân tầng khá phổ biến, nên có một cách ngắn gọn hơn để thực hiện một lần chia duy nhất bằng cách sử dụng hàm `train_test_split()` với đối số `stratify`:

```
strat_train_set, strat_test_set = train_test_split(
    nhà ở, test_size=0.2, stratify=housing["income_cat"],
    random_state=42)
```

Hãy xem liệu điều này có hoạt động như mong đợi hay không. Bạn có thể bắt đầu bằng cách xem xét tỷ lệ các nhóm thu nhập trong tập dữ liệu thử nghiệm:

```
>>> strat_test_set["income_cat"].value_counts() /  
len(strat_test_set)  
3 0.350533  
2 0.318798  
4 0.176357  
5 0.114341  
1 0.039971  
Tên: income_cat, kiểu dữ liệu: float64
```

Với đoạn mã tương tự, bạn có thể đo lường tỷ lệ các loại thu nhập trong...

Bộ dữ liệu đầy đủ. **Hình 2-10** so sánh tỷ lệ các nhóm thu nhập trong

tập dữ liệu tổng thể, trong tập dữ liệu thử nghiệm được tạo ra bằng phương pháp lấy mẫu phân tầng, và trong một

Bộ dữ liệu thử nghiệm được tạo ra bằng phương pháp lấy mẫu ngẫu nhiên hoàn toàn. Như bạn thấy, bộ dữ liệu thử nghiệm.

Kết quả thu được bằng phương pháp lấy mẫu phân tầng có tỷ lệ các nhóm thu nhập gần như bằng nhau.

giống hệt với các dữ liệu trong tập dữ liệu đầy đủ, trong khi tập dữ liệu thử nghiệm được tạo ra bằng cách sử dụng

Việc lấy mẫu hoàn toàn ngẫu nhiên sẽ dẫn đến sai lệch.

	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
Income Category					
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

Hình 2-10. So sánh độ lệch lấy mẫu giữa lấy mẫu phân tầng và lấy mẫu ngẫu nhiên hoàn toàn.

Vì giờ chúng ta sẽ không sử dụng cột `income_cat` nữa nên tốt nhất là xóa nó đi, đưa dữ liệu trở lại trạng thái ban đầu:

```
for set_ in (strat_train_set, strat_test_set):  
    set_.drop("income_cat", axis=1, inplace=True)
```

Chúng tôi đã dành khá nhiều thời gian cho việc tạo tập dữ liệu kiểm thử vì một lý do chính đáng: đây là một phần thường bị bỏ qua nhưng lại vô cùng quan trọng trong một dự án Học máy. Hơn nữa, nhiều ý tưởng này sẽ hữu ích sau này khi chúng ta thảo luận về kiểm định chéo. Giờ là lúc chuyển sang giai đoạn tiếp theo: khám phá dữ liệu.

Khám phá và trực quan hóa dữ liệu để đạt được

Thông tin chi tiết

Cho đến nay, bạn mới chỉ lướt qua dữ liệu để hiểu sơ lược về loại dữ liệu mà bạn đang xử lý. Bây giờ mục tiêu là đi sâu hơn một chút.

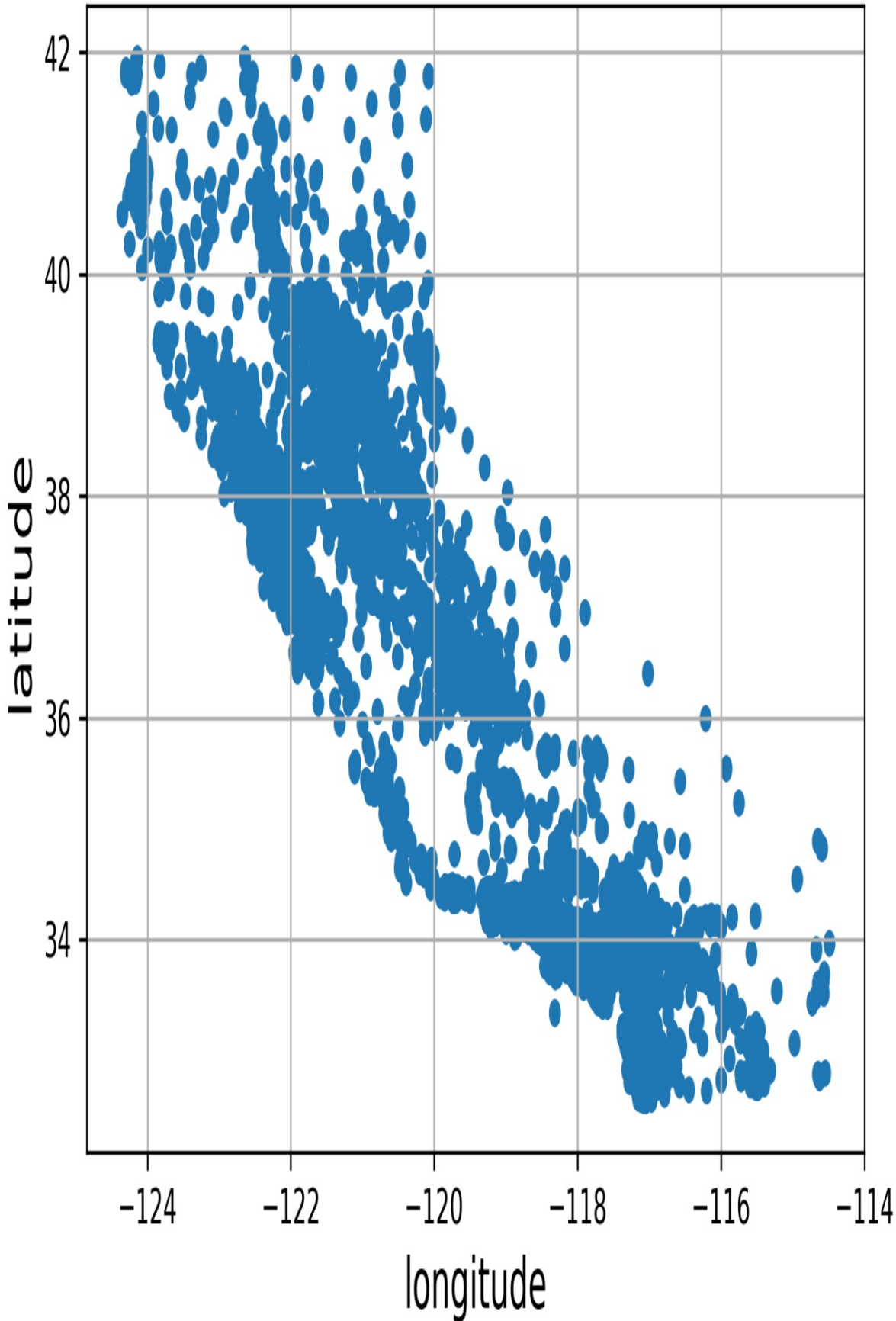
Đầu tiên, hãy đảm bảo bạn đã tách riêng tập dữ liệu kiểm thử và chỉ đang khám phá tập dữ liệu huấn luyện. Ngoài ra, nếu tập dữ liệu huấn luyện rất lớn, bạn có thể muốn lấy mẫu một tập dữ liệu khám phá để việc thao tác dễ dàng và nhanh chóng hơn trong giai đoạn khám phá. Trong trường hợp của chúng ta, tập dữ liệu huấn luyện khá nhỏ, vì vậy chúng ta có thể làm việc trực tiếp trên toàn bộ tập dữ liệu. Vì bạn sẽ thử nghiệm với nhiều phép biến đổi khác nhau trên toàn bộ tập dữ liệu huấn luyện, bạn nên tạo một bản sao của tập dữ liệu gốc để có thể khôi phục lại sau này.

```
housing = strat_train_set.copy()
```

Trực quan hóa dữ liệu địa lý

Vì có thông tin địa lý (vĩ độ và kinh độ), nên việc tạo biểu đồ phân tán của tất cả các quận để trực quan hóa dữ liệu là một ý tưởng hay (Hình 2-11):

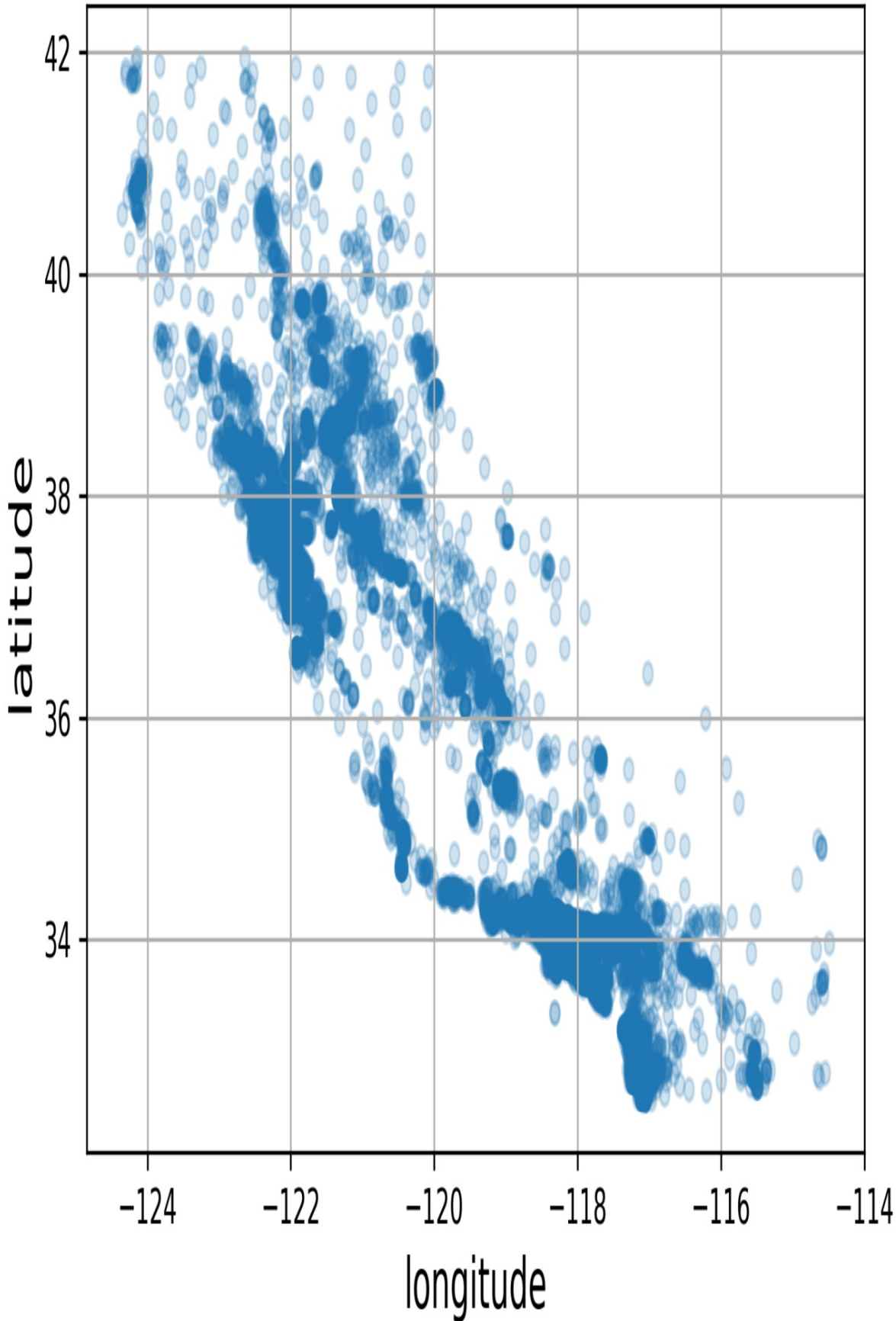
```
housing.plot(kind="scatter", x="longitude", y="latitude",  
grid=True)  
plt.show()
```



Hình 2-11. Biểu đồ phân tán địa lý của dữ liệu.

Hình ảnh này trông giống California, nhưng ngoài ra rất khó để nhận ra bất kỳ mô hình cụ thể nào. Việc đặt tùy chọn `alpha` thành `0,2` giúp dễ dàng hình dung hơn những nơi có mật độ điểm dữ liệu cao (Hình 2-12):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True, alpha=0.2)
plt.show()
```



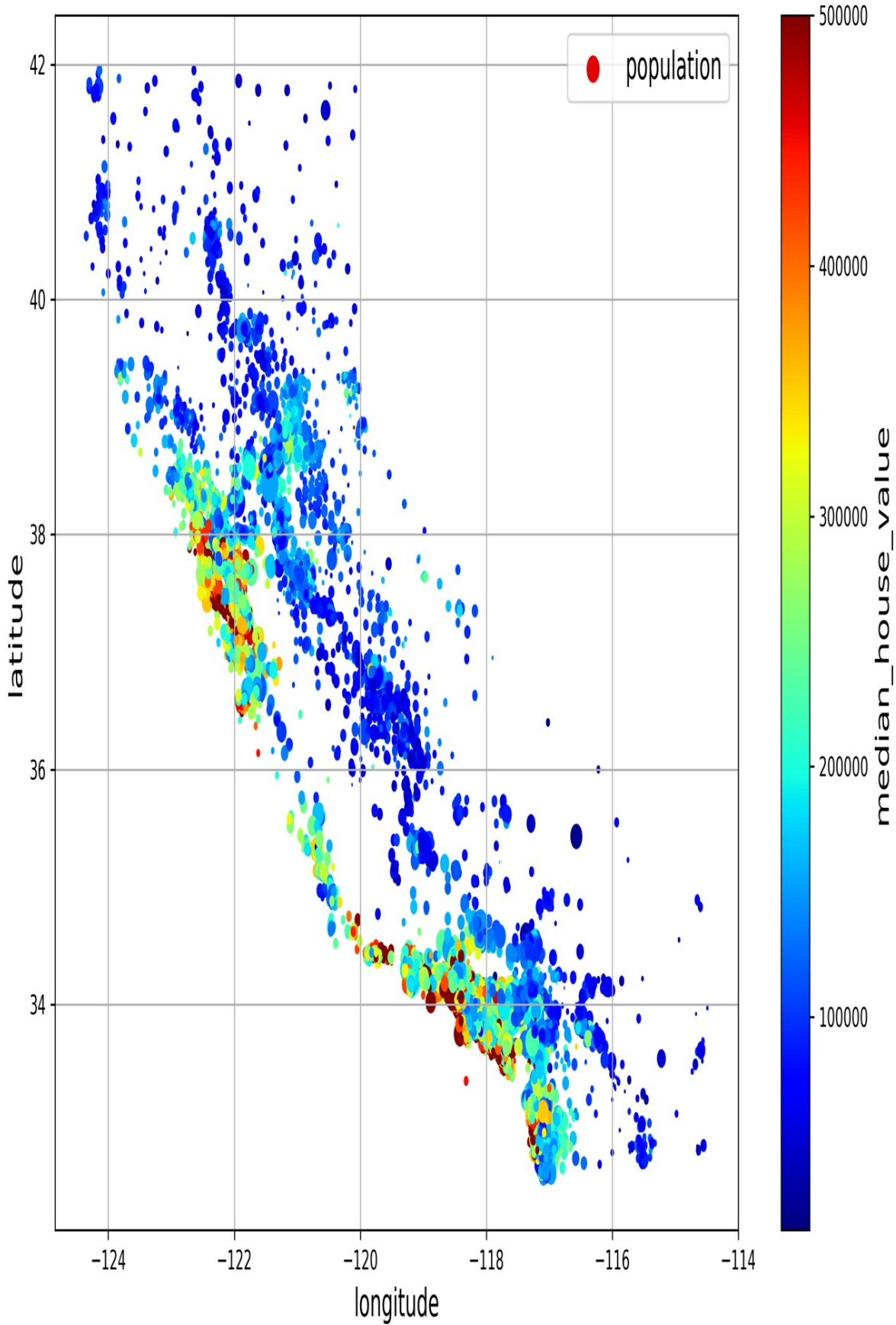
Hình 2-12. Hình ảnh trực quan tốt hơn làm nổi bật các khu vực có mật độ cao.

Giờ thì tốt hơn nhiều rồi: bạn có thể thấy rõ các khu vực có mật độ dân cư cao, cụ thể là Vùng Vịnh và xung quanh Los Angeles và San Diego, cộng thêm một dải dài các khu vực có mật độ dân cư khá cao ở Thung lũng Trung tâm, đặc biệt là xung quanh Sacramento và Fresno.

Não bộ của chúng ta rất giỏi trong việc nhận diện các mẫu hình trong ảnh, nhưng bạn có thể cần điều chỉnh các thông số hiển thị để làm nổi bật các mẫu hình đó.

Bây giờ chúng ta hãy xem xét giá nhà ở (Hình 2-13). Bán kính của mỗi vòng tròn biểu thị dân số của quận (tùy chọn `s`), và màu sắc biểu thị giá cả (tùy chọn `c`). Chúng ta sẽ sử dụng một bản đồ màu được xác định trước (tùy chọn `cmap`) có tên là `jet`, trải dài từ màu xanh lam (giá thấp) đến màu đỏ (giá cao):

```
housing.plot(kind="scatter", x="longitude", y="latitude",
              grid=True,
              s=housing["population"] / 100, label="population",
              c="median_house_value", cmap="jet", colorbar=True,
              legend=True, sharex=False, figsize=(10, 7))
plt.show()
```



Hình 2-13. Giá nhà ở California: màu đỏ là đất, màu xanh là rẻ, các vòng tròn lớn hơn biểu thị các khu vực.
với dân số lớn hơn

Hình ảnh này cho thấy giá nhà ở có mối liên hệ mật thiết với...

vị trí (ví dụ: gần biển) và mật độ dân số, như bạn

Có lẽ đã biết rồi. Thuật toán phân cụm sẽ hữu ích cho

phát hiện cụm chính và bổ sung các tính năng mới để đo lường

khoảng cách đến các trung tâm cụm. Thuộc tính khoảng cách đến đại dương có thể là

Cũng hữu ích, mặc dù ở Bắc California, giá nhà ở ven biển khá cao.

Các quận không quá cao, vì vậy đây không phải là một quy tắc đơn giản.

Tìm kiếm mối tương quan

Vì tập dữ liệu không quá lớn, bạn có thể dễ dàng tính toán tiêu chuẩn.

hệ số tương quan (còn gọi là hệ số tương quan Pearson r) giữa mỗi cặp

các thuộc tính sử dụng phương thức `corr()`:

```
corr_matrix = housing.corr()
```

Bây giờ chúng ta hãy xem xét mức độ tương quan của từng thuộc tính với giá nhà trung bình.

giá trị:

```
>>>
corr_matrix["median_house_value"].sort_values(ascending=False)
giá trị nhà trung bình 1.000000
thu nhập trung vị 0,688380
Tổng số phòng: 0.137455
tuổi trung bình của nhà ở: 0.102175
hộ gia đình 0,071426
tổng số phòng ngủ: 0,054635
dân số -0,020153
kinh độ -0,050859
vĩ độ -0.139584
Tên: median_house_value, kiểu dữ liệu: float64
```

Hệ số tương quan nằm trong khoảng từ -1 đến 1. Khi nó gần bằng 1, thì...

Điều đó có nghĩa là có mối tương quan tích cực mạnh; ví dụ, trung vị

Giá trị nhà ở có xu hướng tăng khi thu nhập trung bình tăng. Khi...

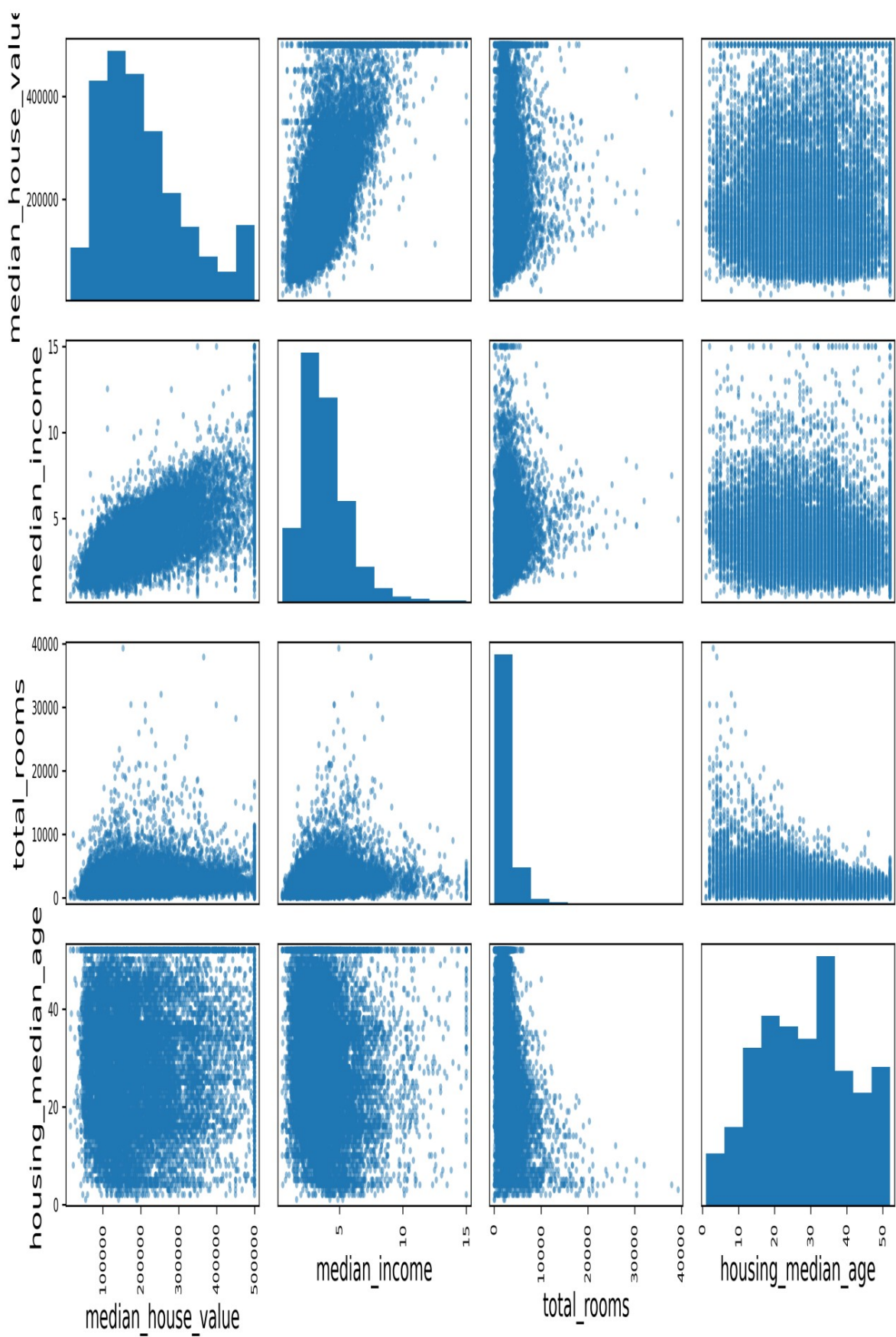
Hệ số gần bằng -1, điều đó có nghĩa là có mối tương quan nghịch mạnh;

Bạn có thể thấy một mối tương quan nghịch nhỏ giữa vĩ độ và giá trị nhà trung bình (tức là giá cả có xu hướng giảm nhẹ khi bạn đi về phía bắc). Cuối cùng, các hệ số gần bằng 0 có nghĩa là không có mối tương quan tuyến tính.

Một cách khác để kiểm tra mối tương quan giữa các thuộc tính là sử dụng hàm `scatter_matrix()` của thư viện Pandas, hàm này vẽ biểu đồ cho mỗi thuộc tính số so với mọi thuộc tính số khác. Vì hiện tại có 11 thuộc tính số, bạn sẽ nhận được $11 \times 11 = 121$ biểu đồ, không thể hiển thị hết trên một trang – vì vậy, chúng ta hãy chỉ tập trung vào một vài thuộc tính có triển vọng nhất dường như có mối tương quan cao nhất với giá trị nhà ở trung bình (Hình 2-14):

```
from pandas.plotting import scatter_matrix

thuộc tính = ["giá trị nhà trung bình", "thu nhập trung bình", "tổng số
phòng", "tuổi trung
    bình của nhà ở"]
scatter_matrix(housing[attributes], figsize=(12, 8)) plt.show()
```

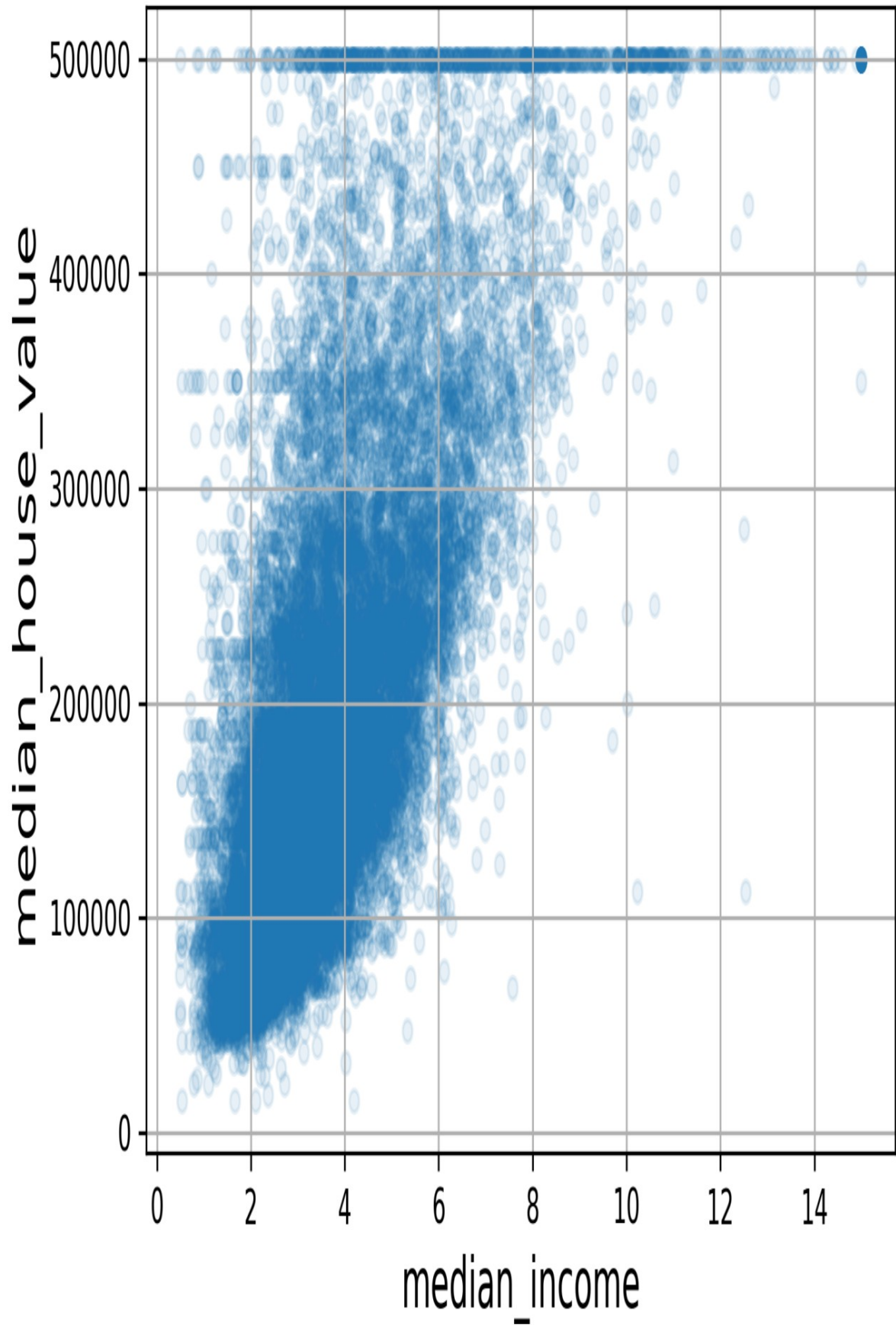


Hình 2-14. Ma trận phân tán này biểu diễn mối quan hệ giữa mỗi thuộc tính số với mọi thuộc tính số khác, cùng với biểu đồ tần suất của các giá trị thuộc tính số trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải).

Nếu Pandas vẽ biểu đồ từng biến với chính nó, đường chéo chính sẽ toàn là các đường thẳng, điều này không hữu ích lắm. Vì vậy, thay vào đó, Pandas hiển thị biểu đồ tần suất của từng thuộc tính (có các tùy chọn khác; xem tài liệu của Pandas để biết thêm chi tiết).

Nhìn vào biểu đồ phân tán tương quan, có vẻ như thuộc tính hứa hẹn nhất để dự đoán giá trị nhà trung bình là thu nhập trung bình, vì vậy hãy phóng to biểu đồ phân tán của chúng (Hình 2-15):

```
housing.plot(kind="scatter", x="median_income",  
y="median_house_value",  
alpha=0.1, grid=True)  
plt.show()
```



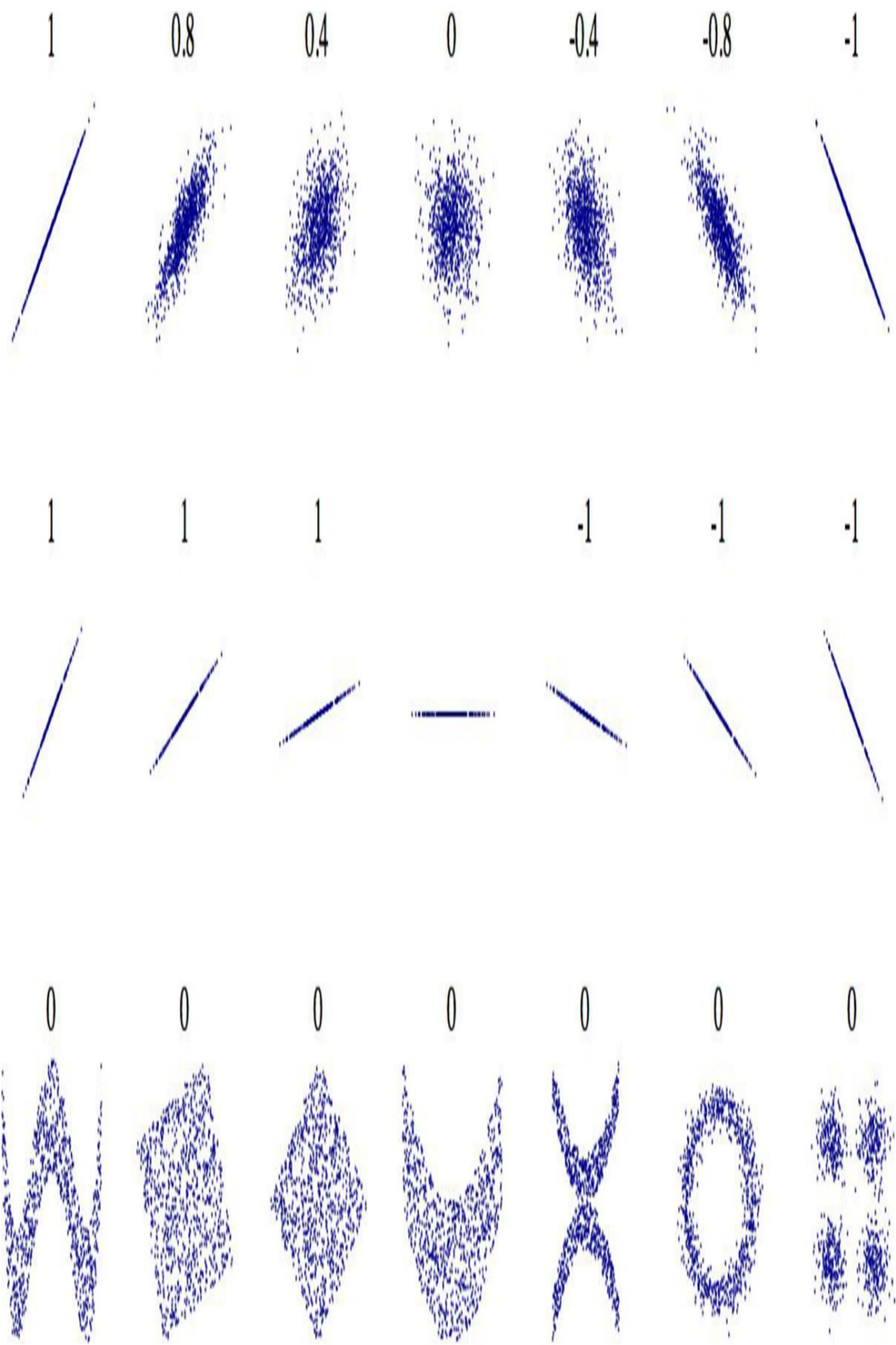
Hình 2-15. Thu nhập trung vị so với giá trị nhà trung vị

Biểu đồ này cho thấy một vài điều. Thứ nhất, mối tương quan thực sự khá mạnh; bạn có thể thấy rõ xu hướng tăng lên, và các điểm không quá phân tán.

Thứ hai, mức giá trần mà chúng ta đã nhận thấy trước đó được thể hiện rõ ràng bằng một đường ngang ở mức 500.000 đô la. Nhưng biểu đồ này còn cho thấy những đường thẳng khác ít rõ ràng hơn: một đường ngang quanh mức 450.000 đô la, một đường khác quanh mức 350.000 đô la, có lẽ một đường quanh mức 280.000 đô la, và một vài đường nữa ở dưới mức đó. Bạn có thể muốn thử loại bỏ các khu vực tương ứng để ngăn thuật toán của bạn học cách tái tạo những điểm bất thường trong dữ liệu này.

CẢNH BÁO

Hệ số tương quan chỉ đo lường các mối tương quan tuyến tính ("khi x tăng, y thường tăng/giảm"). Nó có thể bỏ sót hoàn toàn các mối quan hệ phi tuyến tính (ví dụ: "khi x tiến đến 0, y thường tăng"). Hình 2-16 hiển thị nhiều tập dữ liệu khác nhau cùng với hệ số tương quan của chúng. Lưu ý rằng tất cả các đồ thị ở hàng dưới cùng đều có hệ số tương quan bằng 0, mặc dù trục của chúng rõ ràng không độc lập: đây là những ví dụ về mối quan hệ phi tuyến tính. Ngoài ra, hàng thứ hai hiển thị các ví dụ trong đó hệ số tương quan bằng 1 hoặc -1; lưu ý rằng điều này không liên quan gì đến độ dốc. Ví dụ, chiều cao của bạn tính bằng inch có hệ số tương quan là 1 với chiều cao của bạn tính bằng feet hoặc nanomet.



Hình 2-16. Hệ số tương quan chuẩn của các tập dữ liệu khác nhau (nguồn: Wikipedia; hình ảnh thuộc phạm vi công cộng)

Thử nghiệm các tổ hợp thuộc tính

Hy vọng các phần trước đã cung cấp cho bạn một vài ý tưởng về cách bạn có thể khám phá dữ liệu và thu được thông tin chi tiết. Bạn đã xác định được một vài điểm bất thường của dữ liệu mà bạn có thể muốn làm sạch trước khi đưa dữ liệu vào thuật toán Học máy, và bạn đã tìm thấy các mối tương quan thú vị giữa các thuộc tính, đặc biệt là với thuộc tính mục tiêu. Bạn cũng nhận thấy rằng một số thuộc tính có phân bố lệch phải, vì vậy bạn có thể muốn biến đổi chúng (ví dụ: bằng cách tính logarit hoặc căn bậc hai của chúng). Tất nhiên, kết quả sẽ khác nhau đáng kể tùy thuộc vào từng dự án, nhưng các ý tưởng chung thì tương tự.

Trước khi chuẩn bị dữ liệu cho các thuật toán Học máy, điều cuối cùng bạn nên làm là thử nghiệm các tổ hợp thuộc tính khác nhau. Ví dụ, tổng số phòng trong một quận không hữu ích lắm nếu bạn không biết có bao nhiêu hộ gia đình. Điều bạn thực sự cần là số phòng trên mỗi hộ gia đình. Tương tự, tổng số phòng ngủ đơn thuần cũng không hữu ích lắm: bạn có thể muốn so sánh nó với số phòng. Và dân số trên mỗi hộ gia đình cũng có vẻ là một tổ hợp thuộc tính thú vị để xem xét. Hãy tạo ra các thuộc tính mới này:

```
housing["rooms_per_house"] = housing["total_rooms"] /
housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] /
housing["total_rooms"]
housing["people_per_house"] = housing["population"] /
housing["households"]
```

Và bây giờ chúng ta hãy xem lại ma trận tương quan:

```
>>> corr_matrix = housing.corr()
>>>
corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value      1.000000
median_income           0.688380
rooms_per_house         0.143663
total_rooms             0.137455
```